ChatGPT

Recon47: Repository and Release Plan

Executive Summary

This guide explains how to create and publish the Recon47 tool on GitHub, PyPI, and Docker Hub. It provides step-by-step instructions for setting up the local project (files, setup scripts, Dockerfile, etc.), creating and protecting a GitHub repository, configuring CI/CD with GitHub Actions, and packaging for distribution. We include example commands, file snippets (e.g. `setup.py`, `.github/workflows/*.yml`), tables summarising CI steps, required files, and a final launch checklist. Security best practices are emphasised (using secrets, branch protection, signed commits).

1. Initialize Local Project

1. **Create project structure:**

2. Make a new directory: `mkdir recon47 && cd recon47`.

3. Create core files:

   - `README.md` (project overview).
   - `LICENSE` (e.g. MIT license text).
   - `setup.py` and/or `pyproject.toml` (package configuration).
   - `requirements.txt` (development dependencies).
   - Example config (e.g. `config.yaml`).
   - `Dockerfile` (for building Docker image).
   - Directory for code, e.g. `recon47/`, with `__init__.py` and module files.
   - `tests/` directory for pytest tests.

4. **.gitignore:** Add common ignores (Python, virtual environments, build artifacts). Example:

```
# .gitignore
__pycache__/
*.pyc
.env
venv/
build/
dist/
.DS_Store
```

5. **Example setup.py:**

```
from setuptools import setup, find_packages
setup(
    name='recon47',
    version='0.1.0',
    description='Reconnaissance tool for pentesting',
    author='SHOVON',
    packages=find_packages(),
    install_requires=[
        'requests', 'PyYAML', 'rich', 'tqdm', 'colorama'
    ],
    entry_points={'console_scripts': ['recon47=recon47.cli:main']},
    python_requires='>=3.9',
)
```

6. **Example pyproject.toml:** (if using PEP 517)

```
[build-system]
requires = ["setuptools>=42", "wheel"]
build-backend = "setuptools.build_meta"
```

7. **Dockerfile:** Base image and install steps. Example:

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install --no-cache-dir .
CMD ["recon47", "scan"]
```

8. Add a **CODE_OF_CONDUCT.md**, **CONTRIBUTING.md**, and **ISSUE_TEMPLATE.md** (bug/feature templates). You can use GitHub's templates or examples from reputable repos.

<table> <tr><th>File</th><th>Description</th></tr> <tr><td>README.md</td><td>Project overview, usage instructions, badges.</td></tr> <tr><td>LICENSE</td><td>Open-source license text (MIT, etc.).</td></tr> <tr><td>setup.py / pyproject.toml</td><td>Packaging configuration.</td></tr> <tr><td>requirements.txt</td><td>Dependencies for development (CI, linting, etc.).</td></tr> <tr><td>Dockerfile</td><td>Defines Docker image build.</td></tr> <tr><td>config.yaml</td><td>Example config for Recon47.</td></tr> <tr><td>recon47/</td><td>Python package directory.</td></tr> <tr><td>tests/</td><td>Test suite.</td></tr> <tr><td>.gitignore</td><td>Ignore patterns for Git.</td></tr> <tr><td>CODE_OF_CONDUCT.md</td><td>Community guidelines.</td></tr> <tr><td>CONTRIBUTING.md</td><td>Contribution guidelines.</td></tr> <tr><td>.github/</td><td>GitHub Actions workflows and issue templates.</td></tr> </table>

1. **Initialize Git:**

```
git init
git add .
git commit -m "Initial project scaffold"
```

2. Create GitHub Repository

- **New repo:** On GitHub, create a new repository (public or private as you choose). Name it `recon47`. Do not initialize with README (you have your own).
- **Add remote:**

```
git remote add origin https://github.com/yourusername/recon47.git
git push -u origin main
```

- **Branch Protection:** In the GitHub repo Settings → Branches, add a protection rule for `main`. Require:
- *Require pull request reviews before merging*.
- *Require status checks to pass* (CI build).
- *Require signed commits* (recommend `git config commit.gpgsign true` locally) [1]. (This enforces code review and secure commits.)

3. Configure CI/CD (GitHub Actions)

- **Create workflows:** In `.github/workflows/`, add YAML files.
- **CI Workflow (ci.yml):** On push/pull: install, lint, test, audit. Example:

```
name: CI
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with: python-version: '3.11'
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - name: Lint code
        run: flake8 .
      - name: Run tests
        run: pytest --maxfail=1 --disable-warnings -q
```

```
      - name: Security audit
        run: pip install pip-audit && pip-audit
```

- **Publish Workflow (publish.yml):** On tags, build and publish. Example:

```
name: Publish
on:
  push:
    tags:
      - 'v*'
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Set up Python
        uses: actions/setup-python@v4
        with: python-version: '3.11'
      - name: Install build tools
        run: pip install build twine
      - name: Build distributions
        run: python -m build
      - name: Publish to PyPI
        uses: pypa/gh-action-pypi-publish@v1.5.2
        with:
          user: __token__
          password: ${{ secrets.PYPI_API_TOKEN }}
      - name: Login to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - name: Build and push Docker image
        run: |
          docker build -t yourdockerhubuser/recon47:latest .
          docker push yourdockerhubuser/recon47:latest
```

(Store `PYPI_API_TOKEN`, `DOCKERHUB_USERNAME`, `DOCKERHUB_TOKEN` in **GitHub Secrets**.)

<table> <tr><th>CI Step</th><th>Description</th></tr> <tr><td>Checkout</td><td>Pull code from repository</td></tr> <tr><td>Setup Python</td><td>Install specified Python version</td></tr> <tr><td>Install dependencies</td><td>`pip install -r requirements.txt`</td></tr> <tr><td>Lint</td><td>Run flake8 or pylint</td></tr> <tr><td>Test</td><td>Run pytest suite</td></tr> <tr><td>Audit</td><td>Run `pip-audit` for vulnerabilities</td></tr> <tr><td>Build package</td><td>`python -m build`

creating wheel/sdist</td></tr> <tr><td>Publish</td><td>Use actions to upload to PyPI and Docker Hub</td></tr> </table>

- **Badges:** Add status badges to `README.md` (GitHub Actions, PyPI version, Docker pulls).

4. Packaging and Versioning

- **Semantic Versioning:** Use MAJOR.MINOR.PATCH (e.g. `0.1.0`). Update version in `setup.py` / `pyproject.toml` before release.
- **Changelog:** Maintain `CHANGELOG.md` or a `Unreleased` section in README. For each release, note major changes.
- **Tagging:** After updating version and committing, tag the release:

```
git tag -s v1.0.0 -m "Release v1.0.0"
git push origin v1.0.0
```

The `-s` option signs the tag with GPG (requires GPG setup). Signed tags verify authenticity.

5. Releases

- **Create Release on GitHub:** Go to "Releases" in the repo, draft a new release for tag `v1.0.0`. Add release notes (from CHANGELOG). Attach any built artifacts if needed.
- **Automate via CI:** Our publish workflow will upload packages, but we still recommend creating a GitHub release to document it.
- **Attach Assets:** If needed, attach wheels/sdists or logs to the release page.

6. Documentation and Website

- **README.md:** Include overview, installation (`pip install recon47`), usage examples, links to docs.
- **Docs folder:** Use Sphinx or MkDocs for detailed documentation. Example (MkDocs):
- Install MkDocs: `pip install mkdocs mkdocs-material`.
- Create `mkdocs.yml` and a `docs/` folder with markdown pages.
- Configure GitHub Actions for GitHub Pages. (Enable Pages on `gh-pages` branch or `/docs` folder).
- **Include HTML report demo:** Add a sample HTML report (from Recon47) in `docs/` or root. Link to it in README.

7. Repository Settings and Quality

- **Topics:** Add repo topics like `pentesting`, `reconnaissance`, `python-security`.
- **Badges:** Include build, coverage, PyPI, Docker badges in `README.md`. For example:

```
![CI](https://github.com/youruser/recon47/workflows/CI/badge.svg)
![PyPI](https://img.shields.io/pypi/v/recon47)
![Docker Pulls](https://img.shields.io/docker/pulls/yourdockerhubuser/recon47)
```

- **Branch Protection:** As above, ensure `main` is protected. Require passing checks and PR reviews.
- **Settings:** Set default branch to `main`. Add a CODEOWNERS file if needed.
- **GPG Signing:** Configure `git config --global commit.gpgsign true` to sign commits by default (per GitHub docs [1]).

8. Security Best Practices

- **No Secrets in Repo:** Never commit API keys or passwords. Use GitHub Secrets for tokens (PyPI, Docker Hub).
- **Minimal Permissions:** The GitHub Actions runner uses only `GITHUB_TOKEN` (read/write restricted to this repo). Do not use personal tokens unnecessarily.
- **Code Review:** Enforce reviews for merging (via branch protection).
- **Dependency Audit:** Regularly run `pip-audit` or GitHub's Dependabot to check dependencies.
- **Signed Commits/Tags:** (Optional) Require GPG-signed commits/tags for authenticity [1].

9. Developer Commands and Examples

- **Git Commands:**

```
git clone git@github.com:youruser/recon47.git
cd recon47
git branch -M main
git remote add origin git@github.com:youruser/recon47.git
git push -u origin main
```

- **Local Setup:**

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
pip install -e .       # Install in editable mode
pytest                 # Run tests
```

- **Docker Build/Test:**

```
docker build -t recon47:test .
docker run --rm recon47:test --help
```

10. Example GitHub Actions YAML

- **CI (ci.yml):**

```yaml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with: python-version: '3.11'
      - run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
      - run: flake8 .
      - run: pytest --maxfail=1 --disable-warnings -q
      - run: |
          pip install pip-audit
          pip-audit --exit-zero
```

- **Release (publish.yml):**

```yaml
name: Publish
on:
  push:
    tags: ['v*']
jobs:
  build-and-publish:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: actions/setup-python@v4
        with: python-version: '3.11'
      - run: pip install build twine
      - run: python -m build
      - uses: pypa/gh-action-pypi-publish@v1.5.2
        with:
          password: ${{ secrets.PYPI_API_TOKEN }}
      - uses: docker/login-action@v2
        with:
          username: ${{ secrets.DOCKERHUB_USERNAME }}
          password: ${{ secrets.DOCKERHUB_TOKEN }}
      - run: |
```

```
          docker build -t yourdockerhubuser/recon47:${GITHUB_REF#refs/
tags/} .
          docker push yourdockerhubuser/recon47:${GITHUB_REF#refs/tags/}
```

11. Additional Files

- **.gitignore:** Ignore `__pycache__`, `*.pyc`, `venv/`, `dist/`, etc.
- **CODE_OF_CONDUCT.md:** Use a template (e.g. Contributor Covenant).
- **CONTRIBUTING.md:** Explain how to submit issues/PRs, coding standards.
- **Issue Templates:** In `.github/ISSUE_TEMPLATE/`, add `bug_report.md` and `feature_request.md`.

12. GitHub Pages Demo

- **Enable Pages:** In repo Settings → Pages, set source to `gh-pages` or `main/docs`.
- **Demo Site:** You can publish documentation or host the HTML report. For example, add your HTML report to `docs/` and GitHub will serve it.
- **Embed Sample Report:** Include the HTML report file (as shown in Section 6) in the `docs/` folder. Link to it from the README or docs page.

13. PyPI and Docker Hub Setup

- **PyPI:** Create an account on PyPI. Add the `PYPI_API_TOKEN` (with `workflow` scope) to GitHub Secrets. In `setup.py` ensure metadata (name, version, description) is correct. The GitHub Action will handle uploading.
- **Docker Hub:** Create a repository (e.g. `recon47`). In GH Secrets, add your Docker Hub username and an access token/password as `DOCKERHUB_USERNAME` and `DOCKERHUB_TOKEN`. The CI workflow will log in and push images.
- **Link CI:** In Docker Hub, enable automated builds by connecting to GitHub (optional, since we push from CI).

14. Build and Test Locally

- **Virtual Environment:**

```
python3 -m venv .venv
source .venv/bin/activate
pip install -e .            # installs Recon47 in dev mode
pip install -r requirements.txt
```

- **Run Lint/Test:**

```
flake8 recon47 tests
pytest --maxfail=1 --disable-warnings -q
```

- **Build Distributions:**

```
pip install build
python -m build
# Check dist/ contains .whl and .tar.gz as expected
```

15. Signing and Protection

- **Sign Commits/Tags:** Configure GPG: `git config --global user.signingkey <KEYID>` and `git config --global commit.gpgsign true` [1]. Sign tags with `git tag -s`.
- **Branch Protection:** In GitHub, enforce **Require signed commits** (in addition to reviews and status checks) under branch protection rules.

16. Repository Settings

- **Topics:** Set relevant topics (e.g. `security`, `reconnaissance`, `pentesting`).
- **Badges:** As mentioned, include status and version badges in the README.
- **License:** Ensure `LICENSE` file is present and displayed on GitHub.
- **Default Branch:** Use `main` as default.
- **Code Owners:** (Optional) Add a `CODEOWNERS` file to require PR review by specific teams or individuals.

17. Launch Checklist

<table> <tr><th>Task</th><th>Status</th></tr> <tr><td>All files added and committed</td><td>[ ]</td></tr> <tr><td>README contains installation & usage</td><td>[ ]</td></tr> <tr><td>License file present</td><td>[ ]</td></tr> <tr><td><code>.gitignore</code> configured</td><td>[ ]</td></tr> <tr><td>CI workflows pass on main</td><td>[ ]</td></tr> <tr><td>Branch protection enabled</td><td>[ ]</td></tr> <tr><td>Secrets (PyPI, Docker) set</td><td>[ ]</td></tr> <tr><td>Local tests passing</td><td>[ ]</td></tr> <tr><td>Version bumped and tagged</td><td>[ ]</td></tr> <tr><td>CHANGELOG updated</td><td>[ ]</td></tr> <tr><td>GitHub Release created</td><td>[ ]</td></tr> <tr><td>PyPI & Docker published</td><td>[ ]</td></tr> <tr><td>GitHub Pages (Docs) published</td><td>[ ]</td></tr> </table>

By following these steps and leveraging GitHub's and PyPI's tools, you can create a well-structured Recon47 repository and release process. This ensures reproducible builds, automated testing, and secure distribution of your pentesting tool.

---

[1] Signing commits - GitHub Docs
https://docs.github.com/en/authentication/managing-commit-signature-verification/signing-commits